



Exam Notes - Summary Introduction To Information Systems

Introduction To Information Systems (University of Queensland)



Scan to open on Studocu

MODULE: 0

Functions of DBMS

Enforcing integrity constraints (M2)

- ↳ Domain: INT, string etc
- ↳ Key: No same key
- ↳ Entity Integrity: no NULL Keys
- ↳ Referential: FK deletion/modification
- ↳ Semantic: Rules like X cannot earn more than Y

- ↳ DML - Manage the data
- ↳ DDL - Creating the tables

Logical Data Independence:

Ability to change the conceptual schema w/o external view or program changes

Physical Data Independence:

Ability to change the physical schema w/o changing conceptual schema

MODULE 1: CONCEPTUAL DATA MODELLING THE ENTITY-RELATIONSHIP MODEL

Relationship Degrees:

- 2 Entities: Binary 1:1
- 3 Entities: Ternary 1:1:1
- n Entities: N-ary (3+ Entities)

- @: disjoint - one or the other
- ⊙: overlap - both allowed

MODULE 2: THE RELATIONAL DATA MODEL

Integrity Constraints

Domain: LOL ≠ 4671 assuming that the domain is a 4-digit integer

Key: Cannot have the same key as another tuple

Referential: Foreign key being deleted or modified

Entity Integrity: Keys cannot be NULL

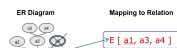
Semantic: UoD rules

↳ "Employee cannot earn more than CEO"

MAPPING 7+1

1. REGULAR ENTITY:

Map all entities and its attributes but LEAVE out the composite and multivalued attributes



2. WEAK ENTITY:

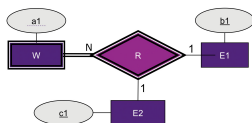
For each weak entity type

1. Create a relation
2. Set its primary key as the combination of the primary keys of owner entities and its own partial key.
3. Include a foreign key from the relation to the primary key of the relation of its owner entity types.
4. Include all simple attributes which are not composite, derived or multivalued.



2. WEAK ENTITY:

For weak entities with multiple owners: e.g.

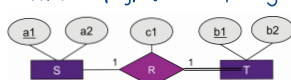


Do the same thing as before but also include the other owner keys

W[a1, b1, c1] W.b1 ref. E1.b1
E1[b1] W.c1 ref. E2.c1
E2[c2]

3. BINARY 1:1 RELATIONSHIP

1. Choose 1 entity (preference to the one with full participation)
2. List its PK and add a FK from the entity relation
3. Include ALL attributes from the relationship type (relationship entity)

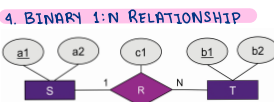


S [a1, a2] T.a1 ref. S.a1
T [b1, b2, a1, c1]

Only weak entities need composite keys
If entity has more than 1 key then only choose 1 that is the most appropriate

4. BINARY 1:N RELATIONSHIP

1. The N-side is the participating entity
2. Do what you normally do. Include its PK and a FK from the relating entity
3. Include all simple attributes from the relationship entity



T is the participating entity bc it's in the N side

S [a1, a2] T.a1 ref. S.a1
T [b1, b2, a1, c1]

5. BINARY M:N RELATIONSHIP

For m:n relationships, map the entities per usual THEN:

- Make a new relation for the Relationship Entity
- Include all FK in the relationship and make them a composite key
- Include all attributes from the relationship entity



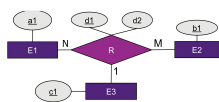
S [a1, a2] R.a1 ref. S.a1
T [b1, b2] R.b1 ref. T.b1
R [a1, b1, c1, c2]

6. MULTIVALUED ATTRIBUTE MAPPING

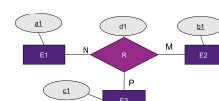
1. Create a new relation for the multivalued (MV) attribute
2. Create a new name: Entity + mv attribute
3. Include the PK of the attribute and the name of the MV attribute



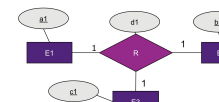
Map out all entities per usual
Create new relation for the relationship attribute
Include all PK from relating entities
Make composite key only for M:N and NOT 1:1 relations
Include all R attributes
m:n:1 Relationship



E1 [a1] R.a1 ref. E1.a1
E2 [b1] R.b1 ref. E2.b1
E3 [c1] R.c1 ref. E3.c1
R [d1, a1, b1, d2, c1]



R [a1, b1, c1, d1]

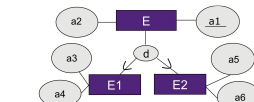


E1 [a1, b1, c1, d1]
E2 [b1]
E3 [c1]

Be careful and
This document is available on

7+1. SUPER AND SUBCLASS

1. Create a new relation for the subclass
2. Have the subclass PK as the superclass PK
3. Include all attributes from the subclass



E [a1, a2]
E1 [a1, a3, a4]
E2 [a1, a5, a6]

Extra Notes to Remember!!

Only steps which use FKs and PKs as

- a composite key
- Weak entities
- Binary M:N relationships
- N-ary relationships

MODULE 3: SQL

DATA DEFINITION LANGUAGE (DDL)

BASIC DDL STATEMENTS

1. DROP TABLE
2. CREATE TABLE
3. ALTER TABLE

ALTER TABLE

DROP TABLE Syntax
DROP TABLE <table name> [CASCADE]
CASCADE - Deletes everything in association
↳ ALSO OPTIONAL

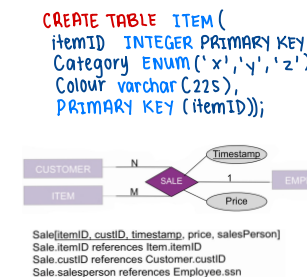
CREATE TABLE Syntax

CREATE TABLE <table name>
(<column name> <column type> [<attribute constraint>]
<column name> <column type> [<attribute constraint>]
<table constraint> [<table constraint>])

columns

CREATE TABLE EXAMPLE

Category
ItemID
Colour
ITEM [itemID, Category, Colour]
CREATE TABLE ITEM (
itemID INTEGER PRIMARY KEY,
Category ENUM('x', 'y', 'z'),
Colour varchar(225),
PRIMARY KEY (itemID));



CREATE TABLE Sale (
itemID INTEGER,
custID INTEGER,
timestamp TIMESTAMP,
price DOUBLE(8,2),
salesPerson Varchar(50)
PRIMARY KEY (itemID, custID, timestamp),
FOREIGN KEY (custID) REF item(custID)
FOREIGN KEY (salesPerson) REF emp(ssn)
↑ references and names have been abrv.

When using CREATE TABLE, make sure to ALWAYS look at the schema and always check at the FK refs.

ALTER TABLE Examples

To add an attribute:
ALTER TABLE Employee
ADD job VARCHAR(12);
Note: values for the added attribute in all tuples will be initially NULL, so NOT NULL must be specified

To drop an attribute:
ALTER TABLE Employee
DROP address;
Note: to drop an attribute which has been used by other tables in their FK references, CASCADE can be used.

To add a constraint:
ALTER TABLE Sale
ADD CONSTRAINT ChkAge
CHECK (price BETWEEN 10 AND 1000);
Note: constraint must have been given a name when it was specified

To drop a constraint:
ALTER TABLE Sale
DROP CONSTRAINT itemid_fk;
Note: constraint must have been given a name when it was specified

DATA MANIPULATION LANGUAGE (DML)

- DML manages the data
- INSERT: add
- UPDATE: change
- SELECT: view

INSERT INTO Syntax

There is more than 1 syntax for insert

Example 1: INSERT from VALUES

INSERT INTO <table name>
VALUES ('x', 'y', 'z')
↑ tuple values

Example 2:

INSERT INTO <table name> (x, y, z)
VALUES ('x', 'y', 'z')
↑ tuple values

DELETE Syntax

DELETE FROM <table name>
WHERE [condition]

UPDATE Statement and Syntax

UPDATE <table name>
SET <column name> = <value expression>

Real-World Example

UPDATE EMPLOYEE
SET Salary = Salary * 1.1
WHERE Name = 'Joyce';

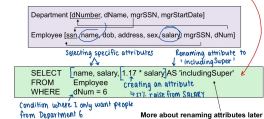
PROJECTION IN SQL

- SELECT DISTINCT <attributes>
- To remove all duplicates
- SELECT * <attributes>
- To select everything in the column selected

IMPORTANT EXAMPLE QUESTION!

Projection and Expressions Example

List the names of all employees working in department 6, their salaries and their salaries with a 17% increase. (1 point)



USING SELECT WITH DATE

Q. List the names and DOB of employees born before 1965
SELECT names, DOB
FROM EMPLOYEE
WHERE DOB < 1965-01-01
↑ DOB < 1965-01-01
↳ Will output some results
↳ Syntax: 'YYYY-MM-DD'

PATTERN MATCHING USING 'LIKE'

Q. Find all movies titles which contain the word 'sin'
SELECT *
FROM MOVIE
WHERE title LIKE '%sin%'
↑ 'sin%' stands for 0 or more arbitrary characters
↑ 'sin%' stands for any one character

Pattern Recognition Examples:

- To find names beginning with 'B': 'B%'
- To find names ending in B: '%B'
- To find names that contain B: '%B%'
- To find names with 5 characters: '_____'

'IN' OPERATOR

The 'IN' operator allows you to specify multiple values in the WHERE clause
E.g. Find customers from Germany, France and Japan

SELECT *
FROM Customers
WHERE Country IN ('Germany', 'France', 'Japan')

'NOT IN' Operator

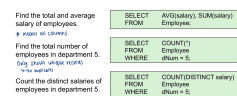
Returns all the records that are NOT in the list of values
E.g. Return customers that are NOT IN Germany, France, Japan
Same query but with 'NOT IN'

'BETWEEN' Operator

... WHERE salary BETWEEN 120000 AND 150000

AGGREGATION AND GROUPING

COUNT: Counts the number of
SUM: Total (numeric value)
MAX/MIN: Highest / Lowest value
AVG: Average



MODULE : 3

IMPORTANT EXAMPLE !!!

Find the total number of employees

```
SELECT COUNT(*)
FROM Employee
```

In each department, count(*) FROM employee. Group Employee count by dNum

```
SELECT dNum, COUNT(*)
FROM Employee
GROUP BY dNum;
```

COUNT(*)

COUNT(*)

Q. Find the AVG salary of employees in each department per gender

```
SELECT AVG(salary), dNum, sex
FROM Employee
GROUP BY dNum, sex;
```

'HAVING' Operator

The 'HAVING' clause follows the 'GROUP BY' clause to add specific conditions

E.g. Find the number of employees in each department which have more than 2 employees

```
SELECT COUNT(*), dNum
FROM Employee
GROUP BY dNum
HAVING COUNT(*) > 2;
```

Q. Find departments which have more than 2 employees earning over \$2,000

```
SELECT COUNT(*), dNum, salary
FROM Employee
WHERE salary > 2000
GROUP BY dNum
HAVING COUNT(*) > 2
```

MULTIPLE RELATIONS SQL QUERIES JOINS IN SQL

```
Department [dNumber, dName, mgrSSN, mgrStartDate]
Employee [ssn, name, dob, address, sex, salary, mgrSSN, dNum]
```

Q. Find the manager names in each department

Equi-Join via WHERE

Joins via Alias and WHERE clause

```
SELECT E.name, D.dname
FROM EMPLOYEE E, DEPARTMENT D
WHERE E.SSN = D.mgrSSN
```

Use ssn as the 'connector' as they are present in both relations

Equi-Join via JOIN

```
SELECT D.dname, E.name
FROM Department D
JOIN Employee AS E
ON D.mgrSSN = E.SSN
```

MORE EXAMPLES

Movie [movieID, title, year]
StarsIn [movieID, starID, role]
MovieStar [starID, name, gender]

Q. Find the names and IDs of all movie stars who have been in a movie

```
SELECT DISTINCT ms.name, s.starID
FROM MovieStar ms
JOIN StarsIn S
ON ms.starID = S.starID
```

Q. Find the names, IDs and characters of all movie star who have been in movieID 1

```
SELECT ms.name, s.starID, s.role
FROM MovieStar ms
JOIN StarsIn S
ON ms.starID = S.starID
WHERE S.movieID = 1
```

COMPLEX JOIN EXAMPLE

Find the ids, names and characters of all movie stars who have been in the movie titled 'Gone with the Wind'.

Movie [movieID, title, year]
StarsIn [movieID, starID, role]
MovieStar [starID, name, gender]

```
SELECT ms.name, s.role, s.starID
FROM MovieStar ms
JOIN StarsIn S
ON ms.starID = S.starID
JOIN Movie M
ON StarsIn.movieID = M.movieID
WHERE M.title LIKE 'Gone with the Wind'
```

Join Example – Recursive Relationship

List the employees who earn less than his/her manager.

When a relation is joined with itself

```
Department [dNumber, dName, mgrSSN, mgrStartDate]
Employee [ssn, name, dob, address, sex, salary, mgrSSN, dNum]
```

Employee and manager - both an employee. Separate them by Employee A and Employee B

RECURSIVE JOINS

Joining a relationship with itself

```
Department [dNumber, dName, mgrSSN, mgrStartDate]
Employee [ssn, name, dob, address, sex, salary, mgrSSN, dNum]
```

Q. List the names of employees who earn less than their managers

```
SELECT A.name AS Employee, A.salary AS EmployeeSalary, B.name AS Manager, B.salary AS ManagerSalary
FROM Employee A
JOIN Employee B
ON A.mgrSSN = B.ssn
WHERE A.salary < B.salary
```

INNER AND OUTER JOIN

Left: Everything on left table rows
Right: Everything on right table rows
Inner: Returns matching values

Full outer: Everything

SET OPERATIONS : UNION

Combines all results
- must be union compatible
↳ Same number of attributes
↳ Same domain
UNION will REMOVE all duplicates
↳ Use DISTINCT to keep duplicates

Movie [movieID, title, year]
StarsIn [movieID, starID, role]
MovieStar [starID, name, gender]

Q. Find IDs of movie stars who have been in a movie in 1944 or 1974

```
SELECT starID
FROM Movie M
JOIN StarsIn S
ON M.movieID = S.movieID
WHERE year = 1944
UNION
SELECT starID
FROM Movie M
JOIN StarsIn S
ON M.movieID = S.movieID
WHERE year = 1974
```

Using just JOIN would keep duplicates

SET OPERATIONS: INTERSECTION

Only returns tuples that are in both R1 and R2

Q. Find IDs of movie stars who have been in a movie in 1944 and 1974

```
SELECT starID
FROM Movie M
JOIN StarsIn S
ON M.movieID = S.movieID
WHERE year = 1944
INTERSECT
SELECT starID
FROM Movie M
JOIN StarsIn S
ON M.movieID = S.movieID
WHERE year = 1974
OR
```

```
SELECT DISTINCT S1.starID
FROM Movie M1
JOIN StarsIn S1 ON M1.movieID = S1.movieID
JOIN Movie M2 ON M2.movieID = S2.movieID
WHERE M1.year = 1944 AND M2.year = 1974
VIA JOIN
```

```
SELECT DISTINCT S1.starID
FROM Movie M1, StarsIn S1,
Movie M2, StarsIn S2
WHERE M1.movieID = S1.movieID AND M1.year = 1944 AND
M2.movieID = S2.movieID AND M2.year = 1974 AND
S2.starID = S1.starID
VIA WHERE
```

SET OPERATIONS: EXCEPT

Returns tuples in R1 but not in R2

Q. Find IDs of movie stars who have been in a movie in 1944 but not in 1974

```
SELECT starID
FROM Movie M
JOIN StarsIn S
ON M.movieID = S.movieID
WHERE year = 1944
EXCEPT
SELECT starID
FROM Movie M
JOIN StarsIn S
ON M.movieID = S.movieID
WHERE year = 1974
```

NESTED QUERIES

A query within a query

Q. Find the name and ID of movie stars who have NOT been in movieID 28

```
SELECT m.name, m.starID
FROM MovieStar M
WHERE m.starID NOT IN (
SELECT s.starID
FROM StarsIn S
WHERE M.movieID = 28);
```

Correlated: Can be computed independently

Non-correlated: Cannot be computed inside out

Non-correlated Example

Q. Find the names of employees who work in a department managed by ssn = 17

```
Department [dNumber, dName, mgrSSN, mgrStartDate]
Employee [ssn, name, dob, address, sex, salary, mgrSSN, dNum]
```

```
SELECT E.name
FROM Employee E
WHERE dNum IN (
SELECT D.dNum
FROM Department D
WHERE D.mgrSSN = 18);
```

Correlated Example

Q. Find the names of departments managed by the same employee

```
SELECT dname, mgrSSN
FROM Department D1
WHERE mgrSSN IN (
SELECT mgrSSN
FROM Department D2
WHERE D1.mgrSSN = D2.mgrSSN
AND D1.dNumber <> D2.dNumber)
```

IMPORTANT TO UNDERSTAND

MODULE:3

ANY/ALL OPERATOR

Just like IN

Q. List department names which have a manager named Jennifer

```
SELECT dname
FROM Department Bc FK
WHERE mgrSSN = ANY (
    SELECT SSN Bc PK for Employee
    FROM Employee
    WHERE name LIKE 'Jennifer');
```

COMPARISON TO ALL EXAMPLE

List all employees whose salary is greater than the salary of those working on department 5

```
SELECT *
FROM Employee
WHERE salary = ANY (
    SELECT salary
    FROM Employee
    WHERE dNum = 5);
```

Single Value Non-Correlated Example

Q. Find the names and salary of employees getting minimum salary

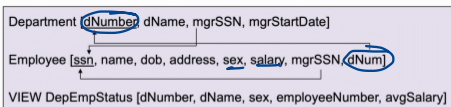
```
SELECT names, salary
FROM Employee
WHERE salary =
    SELECT MIN(salary)
    FROM Employee
```

Cannot use aggregated functions in WHERE clause

Single-Value Correlated Example

Q. Display the name and department number of employees who earn a salary higher than the average in their department

```
SELECT E1.name, E1.dNum
FROM Employee E1
WHERE salary >
    SELECT AVG(salary)
    FROM Employee E2
    WHERE E1.dNum = E2.dNum
```



EXIST / NOT EXIST OPERATOR

- EXIST is a test to find the existence of the data in the subquery
- NOT EXIST will show everything that does not exist in the results

Examples

Q. Find movies that were the only produced that year

```
Movie [movieID, title, year]
StarsIn [movieID, starID, role]
MovieStar [starID, name, gender]
```

```
SELECT *
FROM movie m1
WHERE NOT EXIST (
    SELECT *
    FROM movie m2
    WHERE m1.movieID <> m2.movieID
    AND m1.year = m2.year)
```

Q. Find movies that were not the only movie produced that year

```
SELECT *
FROM movie m1
WHERE EXIST (
    SELECT *
    FROM movie m2
    WHERE m1.movieID <> m2.movieID
    AND m1.year = m2.year)
```

VIEWS

Q. Create a view which shows the count, gender and average salary of employees for each department

CREATE VIEW DepEmpStatus AS

```
SELECT sex, COUNT(*) AS EmployeeNum,
dNumber, dName, AVG(salary)
AS avgSalary
FROM Department D
JOIN Employee E
ON D.dNumber = E.dNum
GROUP BY dNum, sex;
```

```
SELECT *
FROM DepEmpStatus;
```

ANOMALY

Update: When data is partially updated
↳ When data has not been normalised and each row have to be updated

Insert: When new data is added and causes inconsistency

Delete: Removing an attribute that shouldn't be deleted

Module 4 Definitions

Candidate Key (CK)

Have no redundant attributes which identify a tuple in a relation
↳ minimal

Superkey (SK)

A set of attributes that contain all the closure for the relation

Prime Attribute

An attribute that is part of a CK
↳ Non-prime = vice versa

NORMALISATION

1NF: no multivalued or nested relations

customerName	orderNum	items
Tom Jones	123	Hat
Sri Gupta	876	Glass, Pencil

1NF violation

firstName	familyName	orderNum	item
Tom	Jones	123	Hat
Sri	Gupta	876	Glass
Sri	Gupta	876	Pencil

2NF: No partial dependency (PD)

Test
↳ RHS is a prime attribute

$R[A, B, C, D, E]$

$F = \{ABC\} \rightarrow \{DE\}$

$C \rightarrow D$ ← Violates 2NF bc $C \rightarrow D$ is a PD since D can already be retrieved using {ABC}

Normalise to 2NF

Remove redundancy (PD)

Example

Address {houseNum, street, postcode, state, value}

$F = \{\text{houseNum, street, postcode}\} \rightarrow \{\text{state, value}\}$

postcode $\rightarrow$ state

PD in RHS violates 2NF

Specifying Privileges Through Views

If the owner A of a relation R wants another account B to be able to retrieve only some fields of R, then A can create a view V of R

```
Employee [ssn, name, dob, address, sex, salary, mgrSSN, dNum]
```

Suppose that A1 wants to give A3 a limited capability to SELECT from the EMPLOYEE relation and wants to allow A3 to retrieve only the name, dob, and address attributes and only for the tuples with dNum = 5.

```
CREATE VIEW A3EMPLOYEE AS
SELECT name, dob, address
FROM EMPLOYEE
WHERE dNum = 5
```

This document is available on



Downloaded by virin Nguyen (ninhv7307@gmail.com)

MODULE:5

Multilevel Security Example

TC: Tuple Classification
(Highest classified attribute of the tuple)

Example: A multilevel relation to illustrate multilevel security

- The original EMPLOYEE tuples
- Appearance of EMPLOYEE after filtering for classification C users
- Appearance of EMPLOYEE after filtering for classification U users
- Polystantiation of the Smith tuple (a database to maintain multiple records with the same key, in order to prevent inference attacks)

(a) EMPLOYEE (original)

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

(b) EMPLOYEE for classification C (only those with 'S' in TC)

Name	Salary	JobPerformance	TC
Smith U	40000 C	NULL C	C
Brown C	NULL C	Good C	C

(c) EMPLOYEE for classification U (information that is greater than 'U' in TC)

Name	Salary	JobPerformance	TC
Smith U	NULL U	NULL U	U
Brown C	80000 S	Good C	S

(d) EMPLOYEE Polystantiated

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Smith U	40000 C	Excellent C	C
Brown C	80000 S	Good C	S

Too secret (TS) > Secret (S) > Confidential (C) > Unclassified (U)

SQL Support for Role-Based Access Control

Create/drop roles

```
CREATE ROLE manager;
DROP ROLE manager;
```

Grant/revoke privileges

```
GRANT privilegeName
ON objectName
TO {userName | PUBLIC | roleName}
[WITH GRANT OPTION];
```

```
REVOKE privilegeName
ON objectName
FROM {userName | PUBLIC | roleName}
```



Flow control

- Regulates the distribution or flow of information among accessible objects (The Bell-LaPadula Model)
- Verifies information contained in some objects does not flow explicitly or implicitly into less protected objects

Flow policy

- Specifies channels along which information is allowed to move
- Simple form: confidential (C) and nonconfidential (N) ... prohibit flow from C to N

Example:

- Income Tax Computing Service is allowed to retain customer address but not income and deductions data

Hacker behavior

DBMS			
SELECT * FROM Users WHERE username = 'Jack' AND (password = 'wrongpass' OR 'x'='x')			
Id	FullName	Username	Password
1	Jack Smith	Jack	Pass123

Discretionary Access Control

Based on Granting and Revoking privileges

```
CREATE USER <username> IDENTIFIED BY "<password>";
GRANT <privilege> TO <user>; database
```

```
GRANT privilegeName
ON objectName
TO {userName | PUBLIC | roleName}
[WITH GRANT OPTION];
```

```
REVOKE privilegeName
ON objectName
FROM {userName | PUBLIC | roleName}
```

Example: CREATE SCHEMA

```
CREATE SCHEMA AUTHORIZATION oe
CREATE TABLE Product
(color VARCHAR2(10) PRIMARY KEY, quantity NUMBER)
CREATE VIEW RedProduct AS
SELECT color, quantity FROM Product WHERE color = 'RED'
GRANT select ON RedProduct TO hr;
```

The above statement
• creates a schema named oe
• creates the table Product;
• creates the view RedProduct;
• grants the SELECT object privilege on RedProduct to another user hr.

MODULE 5

BCNF - Decomposition

YT: CSDS - Computer and Data Science

Given $R(A, B, C, D, E)$ with the following FD:

$\{AB \rightarrow CD, D \rightarrow E, A \rightarrow C, B \rightarrow D\}$

Find the decomposition to BCNF $X \rightarrow Y$

STEP 1 :

We check to see which FDs of R are not in BCNF, if they are in BCNF then there is no need to decompose R

$\{AB\}^+ = \{A, B, C, D, E\} \rightarrow$ Candidate Key (CK) : AB
 $\hookrightarrow$ To find CK: you're looking for the minimum amount of attributes that their closure gives us all the attributes of the relation R

$D \rightarrow E$ violates BCNF bc according to BCNF, we need $X(D)$ to be the CK of the relation
 $\hookrightarrow$ BC E needs to be dependent on a superkey (SK) and D is not part of a superkey. The SK now is $\{AB\}$

STEP 2 :

We create 2 sub-relations:

- $\{X\}^+ \leftarrow$ The closure of $\{x\}^+$ - which FDs X closes
- $R - \{X\}^+ + X$ - $\{x\}^+ + X \leftarrow$ means X is taken out from $X \rightarrow Y$

Original relation - closure of $\{x\}^+ +$ the attribute X

$AB \rightarrow CD$
 $D \rightarrow E \rightarrow \{D, E\} \rightarrow R_1(DE), R_2(ABCD)$
 $A \rightarrow C$
 $B \rightarrow D$
 The violation minus $D \rightarrow E + D$
 $R_1(AB, C, D, E) = \{x\}^+ + X$

$AB \rightarrow CD$
 $D \rightarrow E$
 $A \rightarrow C$
 $B \rightarrow D$

STEP 3:

For each sub-relation, we check if all of its functional dependencies are now in BCNF

- if they are keep the relation as is
- if not, repeat STEP 2 again for the sub-relation's dependency that does not satisfy BCNF

$R_1(DE)$

CK: D

R_1 FD: $D \rightarrow E$ BCNF

$R_2(ABCD)$

CK: AB

FD: $AB \rightarrow CD$ BCNF

$A \rightarrow C \leftarrow$ NOT BCNF
 $B \rightarrow D \leftarrow$

REPEAT STEP 2 :

$R_3: \{X\}^+$

$R_4: R - \{X\}^+ + X$

$R_3(A, C)$, $R_4(ABD)$

CK = A CK = AB

$A \rightarrow C$ $AB \rightarrow D$

R_3 BCNF R_4 NOT BCNF

$B \rightarrow D$ BCNF

$R_5(BD)$ CK = $B \rightarrow D$

$R_6(AB)$ CK = AB

$B \rightarrow D$ BCNF

$AB \rightarrow AB$ BCNF

Only include fully 'completed' R_s

STEP 4

The final decomposition is composed of sub-relations that all their functional dependencies hold BCNF

$R_1(DE) D \rightarrow E$ $R_3(BD) B \rightarrow D$

$R_2(AC) A \rightarrow C$ $R_4(AB) AB \rightarrow AB$

Only include fully 'completed' R_s

3NF Synthesis

Finding minimal covers for FDs

Minimal Cover Example: Step 3

Step 1:	Step 2:	Step 3:
RHS simplification	LHS simplification	FD set simplification
Every FD has only one attribute on RHS	Remove any redundant attributes from the LHS of each FD	Delete any redundant FDs

- Let's find $\{A, C, D, F\}^+$ with and without considering $\{A, C, D, F\} \rightarrow E$
- $\{A, C, D, F\}^+$ with $\{A, C, D, F\} \rightarrow E = \{A, C, D, F, E, B, G, H\}$
- $\{A, C, D, F\}^+$ without $\{A, C, D, F\} \rightarrow E = \{A, C, D, F, E, B, G, H\}$
- Therefore $\{A, C, D, F\} \rightarrow E$ can be removed.
- $\{A, C, D, F\} \rightarrow G$ can also be removed.

Remove any repeating (redundant) FDs

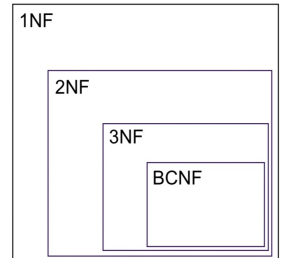
Step 1	Step 2	Step 3
$A \rightarrow B$ $\{A, B, C, D\} \rightarrow E$ $\{E, F\} \rightarrow G$ $\{E, F\} \rightarrow H$ $\{A, C, D, F\} \rightarrow E$ $\{A, C, D, F\} \rightarrow G$	$A \rightarrow B$ $\{A, C, D\} \rightarrow E$ $\{E, F\} \rightarrow G$ $\{E, F\} \rightarrow H$ $\{A, C, D, F\} \rightarrow E$ $\{A, C, D, F\} \rightarrow G$	$A \rightarrow B$ $\{A, C, D\} \rightarrow E$ $\{E, F\} \rightarrow G$ $\{E, F\} \rightarrow H$

Page 100

BCNF $\subset$ 3NF $\subset$ 2NF $\subset$ 1NF

For a relation R with FD set F , for any non-trivial $X \rightarrow A$ in F^+

- 1NF:** identifying non-atomic values from relations
 - Relation should have no multivalued attributes or nested relations
- 2NF:** identifying partial dependencies
 - X is not a proper subset of a candidate key for R , or A is a prime attribute
- 3NF:** identifying partial and transitive dependencies
 - X is a superkey for R , or A is a prime attribute
- BCNF:** identifying all anomalies at the cost of not preserving all FDs
 - X is a superkey for R



Page 74

3NF Example

$R[A, B, C, D, E], F = \{A, B\} \rightarrow C, C \rightarrow D\}$

- Cover already minimal
- Key: $\{A, B, E\}$

Create tables based on $F = \{A, B\} \rightarrow C, C \rightarrow D\}$

- $R_1[A, B, C], R_2[C, D]$

if (any candidate key is missing from the relations)
 add a relation with all prime attributes

- $R_3[A, B, E]$

Remove redundant relations

- Nothing is redundant. Answer is $R_1[A, B, C], R_2[C, D], R_3[A, B, E]$